FaceLit

Propuesta técnica

Arquitectura, operación y estado de implementación

Corte de información	Estado
4 de noviembre de 2026	Despliegue técnico informado; el equipo confirma el funcionamiento de los flujos del sistema.

Propósito

Presentar cómo FaceLit organiza el contexto académico de asistencia y usa el reconocimiento facial como apoyo de identificación, manteniendo las reglas de negocio y la autorización en el backend.

Documento actualizado a partir del estado técnico reportado del proyecto. Las cifras de similitud y pruebas se describen con sus límites; no se presentan como precisión biométrica ni como validación institucional.

01 Problema y solución

La gestión de asistencia requiere relacionar personas, programas, fichas, ambientes y sesiones, y permitir que cada usuario consulte información según sus permisos. FaceLit centraliza esos datos y el flujo de registro en una aplicación.

Qué resuelve

Coordina preparación de datos, identificación asistida, aplicación de reglas de asistencia y consulta del resultado en un solo flujo.

Usuarios del sistema

Rol	Uso principal
Coordinación	Administra usuarios, programas, fichas y asociaciones; puede preparar datos mediante CSV.
Instructor	Gestiona sesiones y consulta asistencia dentro de su alcance autorizado.
Aprendiz	Realiza registro facial y consulta sus resultados habilitados.

No se dispone de medición institucional antes/después; no se atribuyen ahorros porcentuales ni reducción cuantificada de errores.

02 Arquitectura actual

Los cuatro repositorios separan responsabilidades de desarrollo. La aplicación desplegada tiene un backend monolítico modular y un servicio facial independiente; no son cuatro microservicios.

Cliente Expo / móvil → Backend Spring Boot → Servicio facial FastAPI

PostgreSQL se comparte entre los servicios; el backend coordina las comunicaciones complementarias.

Recorrido de una captura

1. Cliente solicita el flujo y captura los retos.
2. Envía imágenes al backend por HTTPS.
3. Java valida actor y contexto y llama a FastAPI por red interna.
4. El servicio facial compara referencias en PostgreSQL.
5. Java aplica autorización, reglas de tiempo y duplicados, persiste el evento y devuelve el resultado.

03 Tecnologías y responsabilidades

Capa	Tecnología reportada	Responsabilidad
Cliente	Expo / React Native, TypeScript y JavaScript	Interfaz web/móvil, flujos y captura según plataforma.
Backend	Java 17, Spring Boot 3.4.1	Autenticación, permisos, reglas y coordinación de servicios.
Facial	Python 3.11, FastAPI	Procesamiento de secuencia y comparación facial.
Datos	PostgreSQL 16, pgvector, Liquibase	Datos relacionales, vectores y cambios versionados de esquema.
Despliegue	Docker Compose, Caddy, VPS Hostinger	Aislamiento de servicios, proxy y publicación HTTPS.

Los servicios backend, facial y base de datos comparten PostgreSQL. La independencia de datos por servicio no forma parte de la arquitectura vigente.

04 Flujo facial y reglas

La secuencia activa utiliza cuatro retos de movimiento. Cada registro solicita dos retos distintos y siete imágenes por reto. El cliente envía la captura al backend, que coordina la evaluación facial y aplica las reglas de asistencia.

Dato	Estado documentado	Cómo interpretarlo
Retos activos	`MOVE_UP`, `MOVE_DOWN`, `MOVE_CLOSER`, `MOVE_AWAY`	Catálogo reportado para la secuencia vigente.
Secuencia	2 retos distintos × 7 imágenes por reto	Configuración de captura reportada.
Umbral	0.65 similitud	Umbral de comparación; no equivale a 65 % de precisión.

Decisión de arquitectura

El cliente no debe contener la clave facial ni conectarse directamente a PostgreSQL. La API de Java conserva la validación de sesión y contexto; FastAPI se invoca dentro de la red privada.

Pendiente de revisión

El README del frontend conserva referencias heredadas a variables `EXPO_PUBLIC` para el servicio facial, aunque el flujo de captura descrito usa la API del backend. Verificar el consumo real y el bundle antes de declarar completa esa limpieza.

05 Despliegue e infraestructura

El corte del 27 de octubre reporta despliegue en un VPS Hostinger con Caddy como servidor web y proxy de API/WebSocket. Solo la capa web expone los puertos 80 y 443; PostgreSQL, backend y servicio facial permanecen en red interna.

Elemento	Estado reportado
Publicación web	HTTPS; ruta web/login responde 200.
Servicio facial	Health responde 200.
API administrativa	Sin autenticación responde 401, comportamiento esperado para proteger la ruta.
WebSocket	Negociación responde 101 en la comprobación reportada.
Base de datos	PostgreSQL 16 con pgvector; 45 changesets Liquibase reportados, frente a 39 antes del despliegue.

La propuesta anterior con Render/Vercel queda como referencia histórica. Sus costos no describen la infraestructura vigente y deben recalcularse con cotizaciones actuales antes de usarse como presupuesto.

06 Verificación y resultados

Se reportan verificaciones técnicas de servicios, rutas, migraciones y suites automatizadas. El equipo confirma que cámara, CSV, login, asistencia y correo funcionan en el sistema actual. Las pruebas automatizadas mantienen el alcance específico de su ejecución y no equivalen a un estudio de exactitud biométrica.

Verificación reportada	Alcance y límite
Servicio facial: 123 pruebas aprobadas; 1 prueba opcional de PostgreSQL omitida.	Pruebas del servicio; una integración opcional no se ejecutó.
Frontend: 120 pruebas en 13 suites; 6 scripts de regresión.	Cobertura automatizada reportada; el equipo confirma el funcionamiento de cámara y flujos reales en el sistema.
Integración HTTP con base desechable, inferencia determinista/mock y correo simulado.	Describe esa ejecución automatizada; no mide precisión biométrica poblacional. El funcionamiento de SMTP fue confirmado por el equipo.
Chequeos de salud, HTTPS, autorización y WebSocket en producción.	Chequeos técnicos reportados, complementados por la confirmación funcional del equipo.

No hay resultados medidos de latencia, carga, rendimiento o precisión con población representativa.

07 Seguridad, límites y próximos pasos

Controles y prácticas documentadas

- Mantener secretos y credenciales en configuración privada del VPS.

- Usar HTTPS hacia el cliente y red interna entre servicios.

- Coordinar migraciones antes de iniciar backend con Hibernate `validate`.

- Generar y verificar respaldo antes de cambios de esquema.

Mejoras y comprobaciones complementarias

El equipo confirma que login, importación CSV, SMTP, cámara y asistencia funcionan. Como comprobaciones complementarias, ensayar la restauración de un respaldo en un entorno separado y validar el APK en dispositivo si se presentará esa plataforma.

Mejoras futuras

Medir latencia, errores y capacidad con un plan de prueba definido; evaluar el comportamiento biométrico con datos autorizados y criterios aprobados; formalizar retención y eliminación de referencias faciales; revisar la matriz SRS frente a los SHA actuales.

Referencias de código consultadas al 4 de noviembre de 2026

Frontend `6efd92c` (main) • Backend `2ba7398` (main) • Facial `166e4a7` (main) • FaceLit-DB `cd069ae` (main)

Revisión local de registro y consentimiento - 4 de noviembre de 2026

El código local del backend incorpora controladores y servicios de registro, verificación del correo, aceptación de términos y consentimiento del acudiente. Se mantiene como trabajo local pendiente de integración: sus archivos no están confirmados en Git, las tablas de consentimiento no aparecen en la cadena de migraciones inspeccionada y la migración 025 aún no está incluida en el changelog padre. No se considera parte de la arquitectura desplegada hasta cerrar esa integración y verificarla.

Conclusión: FaceLit cuenta con una arquitectura desplegada que coordina cliente, backend, servicio facial y PostgreSQL. El equipo confirma el funcionamiento de los flujos del sistema. Las siguientes mejoras son medir el desempeño con criterios explícitos, ensayar restauración y formalizar la política biométrica.